Lab4 无人机控制与竞速实验报告

基本信息	内容
学生	黄博宇，24020036026，中外计科一班，第8组
实验记录	run_20261022_000059
报告日期	2026年10月22日
实验环境	Ubuntu 20.04.6、ROS Noetic、VNAV2021、TESSE Unity

一 实验做了什么

本实验让无人机先在第一道门附近停稳，再按照规划路线完成整条赛道。

可以把系统理解成两个人合作：Lab4的轨迹程序是“路线规划员”，决定每个时刻飞到哪里；Lab3的控制器是“驾驶员”，根据实际位置调整旋翼。

任务	做什么	完成情况
个人题1	求单段轨迹的最优系数和代价	已完成推导和精确核验
个人题2	数清多段轨迹需要哪些连接条件	已完成最小snap及一般r阶推导
首门悬停	飞到第一道门附近并保持	已完成，位置与增益截图已保存
完整竞速	依次跟随赛门目标飞完整条路线	已完成一次完整运行，视频与数据已保存
可选提速	对比多轮运行，寻找更短用时	未开展，不申报最快成绩

一句话记住：先验证“能停稳”，再验证“能沿路线飞”。

二 程序怎样配合

整套系统按“读赛门→画路线→发目标→调旋翼→看反馈”循环工作。

ROS（让不同程序互相传消息的框架）负责连接各个程序。TESSE Unity是提供无人机和赛道的仿真场景。

环节	负责什么	主要文件或消息
读取赛门	从CSV（表格文本）读取并排序位置和朝向	traj_vertices_publisher.cpp
生成路线	把赛门连接成平滑的时间轨迹	trajectory_generation_node.cpp
发送目标	告诉控制器此刻应到哪里、速度多少	/desired_state
执行控制	比较目标与实际状态，计算旋翼命令	controller_node.cpp
返回反馈	把仿真中的实际状态送回ROS	/current_state

本次保留Lab3控制器的简化：**没有加入期望角速度前馈**，即没有额外提前补偿目标转向速度。

一句话记住：Lab4负责“飞哪里”，Lab3负责“怎么飞”。

三 代码补了什么 参数怎样设

本次补全四个Part，使目标从一个固定点变成一条随时间推进的路线。

代码位置	做什么→为什么→结果
Part0	读取第一道门的位姿（位置和朝向），设置零速度和零加速度→先检查控制链路→得到固定悬停目标
Part1.1	读取实际位置和朝向，等待有效状态→避免从错误起点规划→路线从当前位置出发
Part1.2	加入赛门约束，分别优化位置和朝向→让路线经过目标且转向连续→得到平滑轨迹
Part1.3	按经过时间取目标点并发布→让控制器持续获得新目标→无人机沿路线前进

处理朝向时使用yaw（从上往下看时机头朝向的角度）。179°与-179°实际上只差2°；程序先把角度接成连续数值，避免误转一大圈。

参数	本次设置	大白话解释
kx / kv	2 / 3	位置和速度偏离目标时，纠正动作有多强
kr / komega	3 / 1	朝向和转动速度偏离目标时，纠正动作有多强
time_scale	2.0	把估计的每段时间乘2，留出更多跟踪时间
目标发布频率	50Hz，即每秒50次	每隔约0.02秒发送一次目标
位置 / 朝向维度	3 / 1	位置包括XYZ，朝向只规划yaw
N	10	每段10个多项式系数，最高九次
时间估计参数	速度8.0、加速度4.0	用于估算段时间，不是实际飞行硬上限

超过轨迹总时长后，程序保持末端目标。**重跑时要重启跟随节点，单按Unity的R键不会清除旧轨迹。**Lab4使用自己的启动文件，不启动带早期参数的Lab3旧traj_tracking.launch。

四 实验怎样完成

操作顺序的重点是：先接通系统，再开始记录，最后触发飞行。

步骤	做什么	为什么	结果如何
1 准备环境	安装依赖并编译	确认程序能运行	20个包全部编译成功
2 首门测试	启动模拟器、桥接和固定目标	先排除坐标、通信和控制问题	第一门附近能稳定保持
3 切换竞速	停止悬停测试，复位后启动跟随器	避免两套目标同时发送	跟随器等待赛门消息
4 开始记录	同时录屏和录bag	保留出发到结束的完整证据	第二次录制正常保存

步骤	做什么	为什么	结果如何
5 触发飞行	最后启动赛门发布器	航点到达后才开始生成轨迹	完成一次完整赛道运行
6 检查材料	播放视频、检查bag话题	确认保存到了正确内容	视频正常，两个必需状态话题都有数据

bag（按时间保存ROS消息的文件）中，实际状态8365条、期望状态7836条。后续分析只使用轨迹执行段的6708个实际状态采样点。

一句话记住：跟随器→记录器→赛门发布器，顺序不要反。

五 理论题说明了什么

理论部分回答两个问题：怎样选一段轨迹，以及怎样把多段轨迹接平滑。

5.1 单段轨迹

次数提高不一定让代价更低，是否有用取决于约束。

代价是0到1内的速度平方积分，也就是把各时刻的速度平方后累加。QP（二次规划）把多项式系数作为未知量，在端点要求下寻找最小代价。

端点要求	允许的最高次数	最优轨迹P(t)	最小代价J
P(0)=0，P(1)=1	一次	t	1
同上	二次	t	1
同上	三次	t	1
再要求P'(1)=-2	二次	$4t-3t^2$	4
再要求P'(1)=-2	三次	$t/4+9t^2/2-15t^3/4$	17/8=2.125

原端点要求下，直线已经达到最低代价。增加终点速度要求后，直线不再可行，三次轨迹才比二次轨迹更省代价。完整矩阵和验证见deliverable_1.pdf。

5.2 多段轨迹

多段路线不仅要接到同一个位置，还要让连接处的运动变化合适。

snap是位置的四阶导数；最小化它，是为了减少轨迹变化过于猛烈的情况。按一维、各段时间固定、节点位置给定、起终点低阶导数固定的假设计数：

条件	最小snap，r=4	一般r阶
航点位置方程	2k	2k
起终点固定导数	6	2(r-1)
内部低阶导数连续	3(k-1)	(r-1)(k-1)
自然最优条件	3(k-1)	(r-1)(k-1)
完整方程数	8k	2rk

其中k是段数，r是被最小化的导数阶数。“自然最优条件”是求最优解时得到的连接要求，**不能全部算成额外施加的QP约束**。例如r=4、k=3时，有24个系数；显式约束18个，自然条件6个，合计24。

理论最小snap解最高七次；**代码N=10表示最高九次，两者不能直接混用。**完整假设和推导见deliverable_2.pdf。

六 结果怎样 误差主要在哪里

本轮已完成通关，位置跟踪误差较小，主要偏差在高度方向。

6.1 首门悬停

这张状态截图说明当时的位置接近目标，但不能代替整段平均结果。

方向	期望位置m	实际位置m	实际减期望m
X	0.4226201	0.4226182	-0.0000019
Y	3.1652320	3.1652690	0.0000370
Z	2.6700000	2.6304210	-0.0395790

水平位置几乎重合，高度约低3.96厘米。这是一次悬停读数，不是竞速全程的平均高度误差。

6.2 完整竞速

以目标轨迹执行段为准统计，避免把准备和结束后的悬停时间混进去。

分析时先把两种状态消息对到同一时刻。线性插值，就是用前后两个目标值估计中间时刻的目标。随后计算实际位置与期望位置的距离。

RMSE（均方根误差）先把误差平方、求平均，再开方，较大的偏差权重更高。

指标	结果	怎么看
赛门目标数量	12个	不含程序额外加入的当前起点
执行段样本	6708点	用于主要误差统计
三维位置RMSE	**0.056426m，约5.64厘米**	整体位置跟踪误差
最大位置误差	**0.084428m，约8.44厘米**	本执行段内的最大偏差
平均位置误差	0.055555m	实际与目标距离的平均值
X / Y / Z方向RMSE	0.007171 / 0.006392 / 0.055602m	高度误差明显大于水平误差
yaw RMSE	2.2922°	水平朝向的跟踪误差
最高实际速度	1.6292m/s	由实际速度消息计算
实际路径长度	95.570m	相邻位置采样点的距离累计

竞速执行段内，实际高度平均比目标低约5.47厘米。**数据能确认“高度偏低”，但其具体机理尚未验证：[待补充]。**不能仅凭这组数据认定是某个增益或重力参数错误。

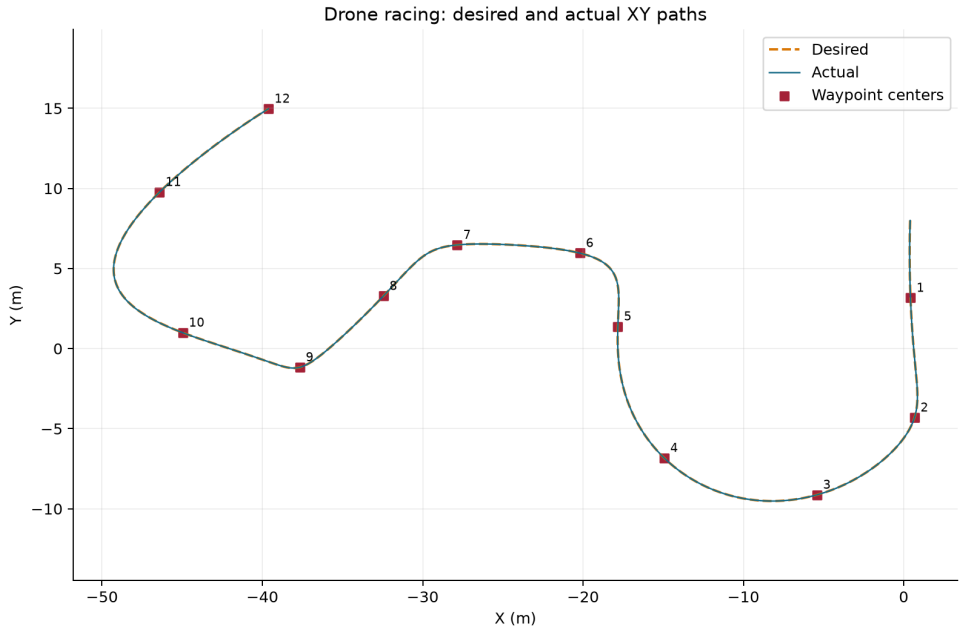


图1 期望与实际水平轨迹

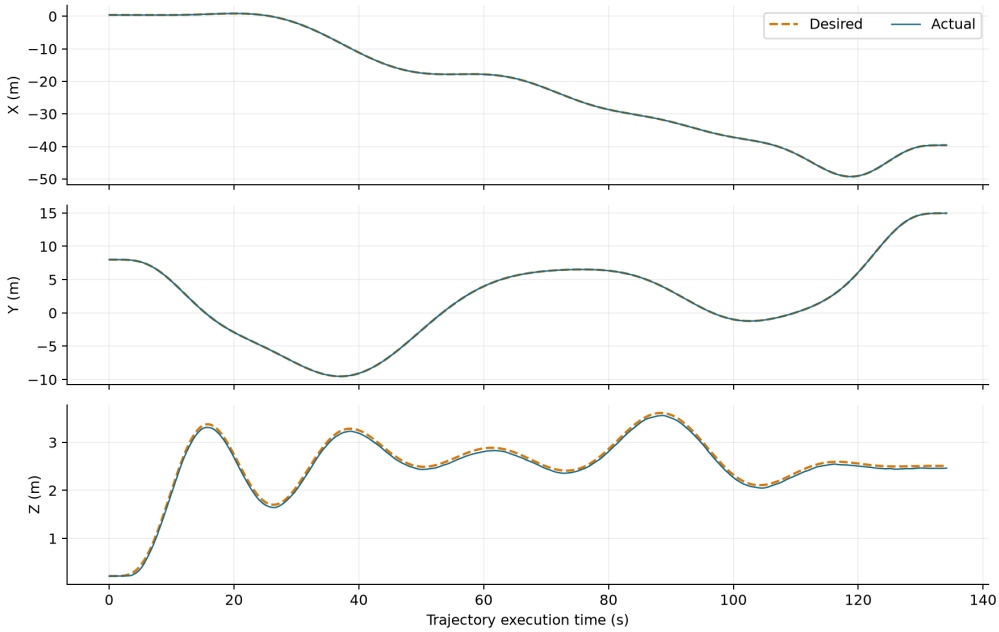


图2 XYZ位置跟踪

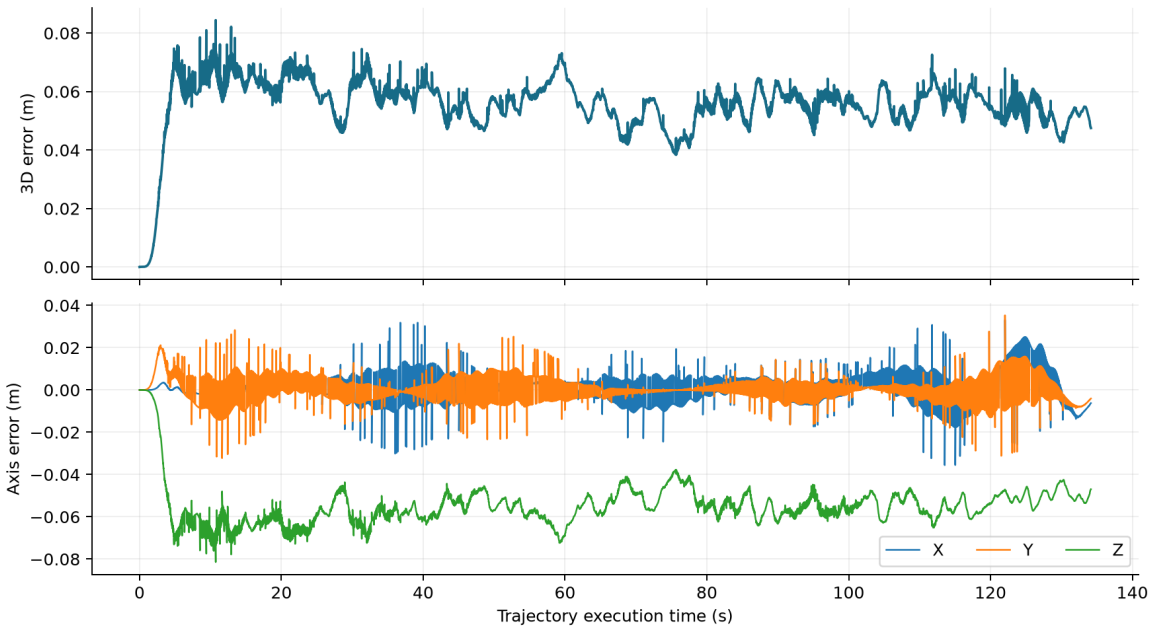


图3 三维与各轴位置误差

6.3 三个时长不要混淆

录了多久、路线安排多久和正式竞速成绩，是三件不同的事。

时长	数值	含义
bag记录	167.28秒	包含准备、飞行和末端保持
视频	160.75秒	录屏文件的播放长度
参考轨迹	134.17秒	目标路线安排的执行时间
独立裁判认定的竞速成绩	[待补充]	本次没有单独的裁判计时记录

一句话记住：误差主要在高度；134.17秒是参考轨迹时长，不是bag时长，也不直接等于裁判成绩。

七 遇到的问题怎样解决

问题主要出在依赖、旧缓存和启动顺序，而不是每次都要重装环境。

问题	做什么→为什么→结果
找不到gflags_catkin	补齐源码依赖→仅安装系统库不够→继续构建
catkin_simple指向旧目录	备份旧build/devel后重建→缓存还记着搬动前的路径→路径问题解决
NLopt下载被拒绝	下载并校验原始安装包，改读本地缓存→避开失效下载链路→全部包编译成功
第一次录制提示Unknown topic	确认跟随器和目标话题就绪后再录→当时无法找到目标话题→第二次录制成功保存

一句话记住：先看错误指向哪一环，再修那一环。

八 实验结论与交付状态

本实验已跑通“赛门输入→路线规划→控制跟随→数据核验”的完整流程。

本轮三维位置RMSE约5.64厘米，最大误差约8.44厘米。高度偏低是主要误差特征，具体原因仍需进一步验证。只有一轮完整材料，因此**不做最快团队或多轮提速结论。**

材料	状态
完整三个ROS代码包	已归档
8张原始截图、参数与bag	已归档
原始视频与连续压缩预览	已归档；原视频分片可完整还原
实验报告与两份理论题PDF	已生成
指标、CSV与复算脚本	已归档
Drive/Dropbox公开的视频与bag链接	**[待补充]**

原视频分片只是把大文件拆开保存，不是剪辑。还原后可用SHA256（文件内容指纹）检查是否与原文件一致。GitHub归档不替代题面指定的Drive/Dropbox链接，也不等于正式提交回执。

数据来自run_20261022_000059；复算入口为data/analysis/analyze_run.py。题目与代码依据：MIT Lab4要求 (https://vnav.mit.edu/labs_2023/lab4/exercises.html)、VNAV2021代码 (https://github.com/MIT-SPARK/VNAV-labs/tree/VNAV2021/lab4)。理论推导、代码准备、分析和文档使用AI辅助，运行、截图和视频来自本人Ubuntu环境。

复习速览

复习时沿着下面七条，就能串起整个实验。

1. **分工：**Lab4规划每个时刻的目标，Lab3控制器调整旋翼跟随。
2. **实现：**Part0首门悬停；Part1.1读状态；Part1.2规划；Part1.3按时间发目标。
3. **朝向：**yaw跨±π要连续展开，避免无谓地绕一整圈。
4. **参数与顺序：**增益2/3/3/1、时间倍率2.0；跟随器→记录器→赛门发布器。
5. **理论：**次数提高不一定更优；多段连接要分清显式约束与自然最优条件。
6. **结果：**一次完整通关，位置RMSE约5.64厘米、最大约8.44厘米；主要偏差在高度。
7. **边界：**134.17秒是参考轨迹时长；没有多轮最快比较，正式媒体公开链接仍待补。